

Documento de Arquitectura y Mecánicas Técnicas de Fighting Games

Especificación técnica, modelos matemáticos, arquitecturas de software, estructuras de datos y máquinas de estados finitas (FSM) para la implementación de Platform Fighters, 2D Tradicionales, Anime/Tag Fighters y 3D Fighters.

Este documento contiene la especificación técnica, modelos matemáticos, arquitecturas de software, estructuras de datos y máquinas de estados finitas (FSM) para la implementación de cuatro estilos fundamentales de juegos de peleas: **Platform Fighters (Brawlhalla)**, **2D Fighters Tradicionales (Street Fighter)**, **Anime/Tag Fighters (Dragon Ball FighterZ)** y **3D Fighters (Tekken)**.

1. BRAWLHALLA (Platform Fighter / 2D Physics Engine)

1.1. Físicas y Variables de Movimiento

Estructuras de Datos y Variables Clave

En un platform fighter 2D, la simulación física combina movimiento determinista basado en *ticks* y modificadores según el estado del personaje.

```
struct CharacterPhysicsState {
    // Vectores de posición y velocidad
    Vector2 position;           // Posición actual en unidades de mundo
    Vector2 velocity;           // Velocidad lineal actual (vx, vy)
    Vector2 external_impulse;    // Impulso acumulado por Knockback/Hits

    // Parámetros de física de movimiento (Constantes/Stats del personaje)
    float run_speed             = 14.5f; // Velocidad horizontal máxima en tierra
    float air_speed              = 11.0f; // Velocidad horizontal máxima en aire
    float run_acceleration       = 85.0f; // Aceleración horizontal terrestre
    float air_acceleration       = 60.0f; // Aceleración horizontal aérea
    float friction_ground        = 70.0f; // Desaceleración en tierra sin input
    float friction_air           = 12.0f; // Arrastre/fricción aérea habitual

    // Caída y Gravedad
    float gravity_accel          = -45.0f; // Aceleración por gravedad constante
    float max_fall_speed         = -20.0f; // Velocidad terminal de caída normal
    float fast_fall_speed        = -38.0f; // Velocidad terminal en Fast Fall
    float fast_fall_impulse      = -15.0f; // Impulso inicial al activar Fast Fall

    // Contadores de recursos
    int jumps_max                = 3;      // Salto principal + 2 saltos aéreos
    int jumps_remaining          = 3;
    bool has_recovery             = true;   // Ataque de recuperación aéreo disponible
    bool has_ground_pound        = true;   // Ataque pesado hacia abajo en aire

    // Cooldowns y Timers (medidos en Frames/Ticks a 60Hz)
    int dodge_cooldown           = 0;      // Cooldown de esquivar (aire vs tierra)
    int wall_touch_count         = 0;      // Contador para activación de Wall Slip
}
```

```
bool is_touching_wall = false;
bool is_grounded      = false;
bool is_fast_falling  = false;
};
```

Fórmula y Lógica del Fast Fall

El *Fast Fall* no modifica la gravedad global, sino que fuerza la velocidad terminal a un límite más alto mediante la aplicación de un impulso instantáneo y un cambio en la velocidad de corte (*clamp*).

Condición de Activación

Se activa cuando el jugador presiona la dirección **Abajo** (**Input.y < -0.7f**) mientras se encuentra en fase descendente en el aire (**velocity.y < 0.0f**) y no está en estado de *Stun* ni ejecutando un ataque no cancelable.

Fórmula: Si ($v_y < 0$) AND ($Input_y < -0.7$) AND NOT $is_fast_falling \Rightarrow \begin{cases} v_y = \min(v_y + fast_fall_impulse, fast_fall_speed) \\ is_fast_falling = true \end{cases}$

Lógica de integración por Tick (60 Hz):

```
void UpdateVerticalPhysics(CharacterPhysicsState& p, float deltaTime) {
    if (!p.is_grounded) {
        // Aplica aceleración de gravedad
        p.velocity.y += p.gravity_accel * deltaTime;

        // Limita la velocidad de caída según si está en Fast Fall
        float terminal_limit = p.is_fast_falling ? p.fast_fall_speed : p.max_fall_speed;
        if (p.velocity.y < terminal_limit) {
            p.velocity.y = terminal_limit;
        }
    } else {
        p.velocity.y = 0.0f;
        p.is_fast_falling = false;
    }
}
```

Mecánica de Gravity Cancel (GC)

El *Gravity Cancel* es la cancelación de un *Air Dodge* neutral o direccional ejecutando la entrada de un ataque terrestre durante los primeros frames de invulnerabilidad de la esquivas. Esto congela la gravedad y otorga acceso a ataques grounded en el aire.

Parámetros Técnicos

- **Ventana de ejecución (GC Window):** Frames \$1\$ al \$4\$ del *Air Dodge*.
- **Aceleración congelada:** \$g = 0\$ y \$v_y = 0\$ durante los frames activos del movimiento activado por GC.
- **Penalización de recurso:** Consume el *Dodge* aéreo y aplica el cooldown extendido en aire (\$180\text{ frames} = 3\text{ segundos}\$).

Diagrama de Secuencia FSM (Gravity Cancel):

```
State: AIRBORNE
    Trigger: PRESS_DODGE
        State: AIR_DODGE (Invulnerable = true, DodgeTimer = 0)
            [DodgeTimer <= 4 Frames] AND (PRESS_ATTACK)
```

```

■ ■ ■ ■ TransitionTo: GRAVITY_CANCEL_ATTACK
■ ■ ■ ■ Velocity.x = 0, Velocity.y = 0
■ ■ ■ ■ ApplyGravity = false
■ ■ ■ ■ ExecuteGroundedAttackAnimation()
■ ■ ■ ■ SetDodgeCooldown(180_frames)
■ ■ ■ ■
■ ■ ■ ■ [DodgeTimer > 4 Frames]
■ ■ ■ ■ Continúa AIR_DODGE normal
    
```

Gestión de Recursos Aéreos y Mecánica de Wall Slip

Para evitar el acampe pasivo en paredes (*wall hugging*), se utiliza el sistema **Wall Slip**.

```

+-----+
|  AIRBORNE STATE  |
+-----+
      |              ^
Wall Contact |              | Touch Ground
(vy < 0)     v              | (Resets All)
+-----+
|  WALL TOUCH / SLIDING|
|  (WallTouches++)    |
+-----+
      |
      | WallTouches >= 15
      v
+-----+
|  WALL SLIP STATE    |
|  (No Jumps, No Recovery|
|  Fast Fall Forced)   |
+-----+
    
```

Reglas de Lógica de Recursos

```

void HandleWallInteraction(CharacterPhysicsState& p, bool touching_wall) {
    if (touching_wall && !p.is_grounded && p.velocity.y < 0) {
        p.is_touching_wall = true;

        // Deslizar por la pared resetea la Recovery pero consume un 'Wall Touch'
        p.has_recovery = true;
        p.has_ground_pound = true;

        if (p.wall_touch_count >= 15) {
            // WALL SLIP ACTIVADO: El personaje pierde la capacidad de saltar en pared
            p.jumps_remaining = 0;
            p.velocity.y = p.fast_fall_speed; // Cae instantáneamente
        }
    } else if (p.is_grounded) {
        // El contacto directo con la plataforma principal limpia todos los contadores
        p.jumps_remaining = p.jumps_max;
        p.has_recovery = true;
        p.has_ground_pound = true;
        p.wall_touch_count = 0;
    }
}
    
```

1.2. Mecánicas de Daño y Knockback

Fórmula de Impulso y Knockback Total (\$K\$)

El Knockback generado por un golpe es una función vectorial donde el daño acumulado del personaje que recibe el impacto escala exponencialmente la distancia y aceleración del despeje.

Ecuación Matemático-Física:

Fórmula: $K = [(\{Damage_target + Damage_attack\}10 + (\{Damage_target \times Damage_attack\}20)) \times (\{200\{Weight_target + 100\} \times 1.4 \} + BaseKnockback$

Fórmula: $K_{final} = K \times KnockbackScaling$

Donde:

- $\{Damage_target\}$: Porcentaje acumulado por la víctima (\$0.0\$ a \$300.0+\$).
- $\{Damage_attack\}$: Daño base de la caja de golpe (*Hitbox*).
- $\{Weight_target\}$: Atributo de masa del personaje (rango típicamente entre \$75\$ y \$115\$).
- $\{BaseKnockback\}$ (\$BK\$): Impulso plano fijo asignado al ataque.
- $\{KnockbackScaling\}$ (\$KS\$): Multiplicador de escalado por daño del movimiento.

Cálculo del Vector de Trayectoria de Despeje:

El ángulo del golpe (θ) viene codificado en la propiedad del *Hitbox*. El vector de velocidad final asignado al receptor en la fase de impacto es:

Fórmula: $\vec{V}_{impact} = \begin{bmatrix} K_{final} \times \cos(\theta) \\ K_{final} \times \sin(\theta) \end{bmatrix}$

Vector de Trayectoria y Duración de Stun

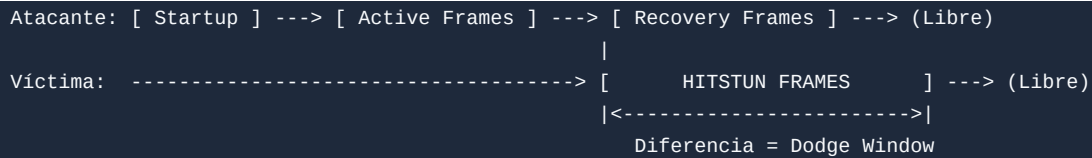
Cuando un personaje es impactado, entra en un estado de **Hitstun** durante el cual sus inputs de control quedan bloqueados y se desactiva la colisión física normal.

```
struct HitstunData {
    float initial_knockback;
    float trajectory_angle_rad;
    int   stun_frames;

    // Cálculo de frames de stun basados en el Knockback total
    static int CalculateStunFrames(float total_knockback, float move_stun_base) {
        // Fórmula de duración de inmovilización:
        // StunFrames = BaseStun + (TotalKnockback * StunScalingFactor)
        const float STUN_SCALING_FACTOR = 0.055f;
        return static_cast<int>(move_stun_base + (total_knockback * STUN_SCALING_FACTOR));
    }
};
```

Matriz de True Combos vs. Strings (Dodge Window)

La distinción entre un **True Combo** y una **String** depende de los frames de ventaja resultantes tras el *Recovery* del atacante frente al *Hitstun* del receptor.



Regla Matemática para Detección de True Combo vs String:

Sea F_{hitstun} la cantidad de frames que el receptor permanece en estado de stun tras el impacto, F_{recovery} los frames restantes del ataque del atacante tras conectar, y F_{startup2} los frames de inicio del siguiente ataque:

Fórmula: $DodgeWindow = F_{\text{startup2}} - (F_{\text{hitstun}} - F_{\text{recovery}})$

- Si $\text{DodgeWindow} \leq 0$ $\implies \mathbf{True\ Combo}$ (Imposible esquivar, el segundo ataque conecta antes o en el frame en que termina el Hitstun).
- Si $\text{DodgeWindow} > 0$ $\implies \mathbf{String}$ (La víctima tiene N frames libres donde el botón de Dodge será procesado antes de ser golpeado nuevamente).

1.3. Máquina de Estados Finita (FSM) del Personaje



Pseudocódigo de la FSM Principal

```

enum class CharacterState {
    Grounded,
    Crouching,
    Airborne,
    AttackGrounded,
    AttackAirborne,
    AirDodge,
    Hitstun,
    WallSliding
};
    
```

```

class CharacterFSM {
public:
    CharacterState current_state = CharacterState::Grounded;
    CharacterPhysicsState physics;

    void ProcessInputAndTransitions(const InputBuffer& input) {
        switch (current_state) {

            case CharacterState::Grounded:
                if (!physics.is_grounded) {
                    TransitionTo(CharacterState::Airborne);
                    break;
                }
                if (input.IsPressed(InputButton::Jump)) {
                    physics.velocity.y = 18.0f; // Salto inicial
                    physics.jumps_remaining--;
                    TransitionTo(CharacterState::Airborne);
                    break;
                }
                if (input.IsPressed(InputButton::LightAttack) || input.IsPressed(InputButton::HeavyAttack))
                {
                    TransitionTo(CharacterState::AttackGrounded);
                    break;
                }
                if (input.Y < -0.5f) {
                    TransitionTo(CharacterState::Crouching);
                    break;
                }
                break;

            case CharacterState::Airborne:
                if (physics.is_grounded) {
                    TransitionTo(CharacterState::Grounded);
                    break;
                }
                if (physics.is_touching_wall) {
                    TransitionTo(CharacterState::WallSliding);
                    break;
                }
                // Detección de Air Dodge / Gravity Cancel
                if (input.IsPressed(InputButton::Dodge) && physics.dodge_cooldown == 0) {
                    TransitionTo(CharacterState::AirDodge);
                    break;
                }
                // Consumo de saltos aéreos
                if (input.IsJustPressed(InputButton::Jump) && physics.jumps_remaining > 0) {
                    physics.velocity.y = 16.0f; // Salto aéreo
                    physics.jumps_remaining--;
                }
                // Trigger de Fast Fall
                if (input.Y < -0.7f && physics.velocity.y < 0.0f) {
                    physics.is_fast_falling = true;
                }
                break;
        }
    }
}

```

```

        case CharacterState::AirDodge:
            // Evaluamos si el jugador ejecuta un Gravity Cancel
            if (dodge_timer <= 4 && (input.IsPressed(InputButton::LightAttack) || input.IsPressed(InputButton::HeavyAttack))) {
                physics.velocity = Vector2(0.0f, 0.0f); // Congelar inercia
                physics.dodge_cooldown = 180;           // Cooldown de 3s
                TransitionTo(CharacterState::AttackGrounded); // Ejecuta ataque terrestre en el aire
                break;
            }
            if (dodge_timer > 16) { // Fin de los frames de la esquiv a aérea
                TransitionTo(CharacterState::Airborne);
            }
            break;

        case CharacterState::Hitstun:
            // El control del jugador está completamente bloqueado hasta que el temporizador llegue a 0
            if (hitstun_timer <= 0) {
                if (physics.is_grounded) {
                    TransitionTo(CharacterState::Grounded);
                } else {
                    TransitionTo(CharacterState::Airborne);
                }
            }
            break;

        default:
            break;
    }
}

private:
    int dodge_timer = 0;
    int hitstun_timer = 0;

    void TransitionTo(CharacterState new_state) {
        current_state = new_state;
    }
};

```

1.4. Tabla de Parámetros Globales (Brawlhalla)

| Parámetro | Valor Predeterminado | Unidad | Descripción |

| :--- | :--- | :--- | :--- |

| **Tick Rate** | 60 | Hz | Frecuencia de actualización de la lógica del motor de física y FSM |

| **Base Weight** | 100.0 | Puntos | Masa estándar de referencia para la fórmula de Knockback |

| **Ground Dodge Frames** | 20 | Frames | Duración de invulnerabilidad de la esquiv a en tierra |

| **Air Dodge Frames** | 16 | Frames | Duración de invulnerabilidad de la esquiv a en el aire |

| **Ground Dodge CD** | 70 | Frames | Tiempo de recarga del Dodge tocando superficie (~1.16s) |

| **Air Dodge CD** | 180 | Frames | Tiempo de recarga del Dodge ejecutado en el aire sin tocar tierra (3.0s) |

| **GC Window** | 1 a 4 | Frames | Ventana de cancelación de Air Dodge a ataque terrestre |

| **Wall Slip Threshold** | 15 | Taps | Límite de contactos continuos con pared sin tocar tierra firme |

2. STREET FIGHTER (Classic 2D / Deterministic Frame Data)

2.1. Motor de Frame Data y Ciclo de Vida del Ataque

El motor de Street Fighter opera de forma strictly determinista a **60 FPS** (16.66 ms por frame). La vida de cualquier movimiento se divide en tres fases secuenciales representadas por una estructura de datos rígida.

```
struct AttackFrameData {
    int startup_frames;        // Frames desde el input hasta el primer frame activo
    int active_frames;        // Frames donde la Hitbox está presente y puede impactar
    int recovery_frames;      // Frames de vulnerabilidad tras finalizar las active frames

    int hit_advantage;        // Ventaja de frames en impacto (+/-)
    int block_advantage;      // Ventaja de frames en bloqueo (+/-)

    int cancel_window_start;  // Frame donde inicia la ventana para Special Cancel
    int cancel_window_end;    // Frame donde finaliza la ventana para Special Cancel

    bool forces_crouch;       // Fuerza al rival a estado agachado
    bool is_overhead;         // Rompe guardia baja (debe bloquearse de pie)
};
```

Cálculo del Frame Advantage (Matemática de Prioridad)

El **Frame Advantage** determina quién recupera el control de su personaje antes tras un impacto o bloqueo.

Fórmula: $FrameAdvantage_{Hit} = VictimHitstunFrames - AttackerRecoveryFrames$

Fórmula: $FrameAdvantage_{Block} = VictimBlockstunFrames - AttackerRecoveryFrames$

- Si $FrameAdvantage > 0$ (+): El atacante actúa antes. Si es +3 en Hit, un ataque con 3 frames de *Startup* conectará automáticamente como **Link**.
- Si $FrameAdvantage < 0$ (-): El atacante queda vulnerable. Si un movimiento queda en -6 en Block, cualquier ataque rival con 6 frames de *Startup* garantiza un **Punish** (Castigo).

2.2. Lógica de Cancelación de Animaciones (Special & Super Cancel)

Un *Cancel* interrumpe los *Active* o *Recovery* frames de un golpe normal e inicia inmediatamente el *Startup* de un ataque especial.

```
void ProcessAttackCancels(CharacterFSM& attacker, const InputBuffer& buffer) {
    AttackFrameData current_move = attacker.current_attack;
    int current_frame = attacker.attack_current_frame;

    // Verificar si el frame actual se encuentra en la ventana permitida
    bool in_cancel_window = (current_frame >= current_move.cancel_window_start) &&
        (current_frame <= current_move.cancel_window_end);

    if (in_cancel_window && attacker.has_connected_hit) {
        // Verificar si hay un movimiento especial válido en el Búfer de Entradas
    }
```



```
SpecialMove input_special = buffer.CheckSpecialMoveBuffer();

if (input_special != SpecialMove::None) {
    // Cancelar animación actual
    attacker.StopCurrentAnimation();
    attacker.ResetHitboxes();

    // Transición inmediata al nuevo ataque
    attacker.ExecuteSpecialMove(input_special);
}
}
```

2.3. Sistema de Recursos: Drive Gauge & Burnout (Ejemplo SF6)

Lógica de Estados del Drive Gauge



Modificadores de Físicas y Frame Data en Burnout:

- 1. **Penalización de Blockstun:** Todos los ataques recibidos en bloqueo causan **+\$4\$ frames adicionales de Blockstun** para el jugador en **Burnout**.
- 2. **Chip Damage:** Permite morir por daño en bloqueo (**Chip Death**) ante ataques especiales.
- 3. **Pérdida de opciones:** Se desactivan las mecánicas **Drive Rush**, **Drive Impact** y **Drive Parry**.

3. DRAGON BALL FIGHTERZ (High-Speed Anime Fighter)

3.1. Movimiento Vectorial: Homing Super Dash & Vanish

Lógica de Traza Vectorial de Super Dash

El **Super Dash** calcula dinámicamente un vector de persecución hacia la posición central de la **Hurtbox** rival, reajustando su dirección cada frame con una tasa de giro angular máxima (ω_{max}).

```
Vector2 CalculateSuperDashVelocity(Vector2 current_pos, Vector2 target_pos, float speed, float max_turn_rate, float deltaTime) {
    Vector2 desired_direction = (target_pos - current_pos).Normalized();

    // Interpolar la dirección actual hacia la dirección deseada (Limitación de giro)
    Vector2 new_direction = Vector2::Slerp(current_direction, desired_direction, max_turn_rate * deltaTime);

    return new_direction * speed; // Retorna la velocidad vectorial resultante
}
```

- **Regla de Prioridad:** Si el rival ejecuta un \$2H\$ (Heavy Abajo), el ataque \$2H\$ otorga **invulnerabilidad contra atributos aéreos/dashes** desde el frame 4, destruyendo el *Super Dash* en el impacto.

Lógica de Posicionamiento de Vanish (Teleport)

Consumes 1 barra de Ki (\$1.0\$ bar). Calcula la posición exacta detrás de la víctima invirtiendo su vector de mirada (**FacingDirection**).

Fórmula: $\text{vec}\{P_{\text{vanish}} = \text{vec}\{P_{\text{victim}} + (\text{vec}\{D_{\text{facing}} \times -d_{\text{offset}})\}$

3.2. Sistema de Combos: Escalado y Degradación (Combo Scaling)

Para evitar combos infinitos, el motor aplica una degradación doble: **Reducción de Daño** y **Reducción de Hitstun**.

Fórmulas de Escalado de Daño

Fórmula: $D_{\text{actual}} = D_{\text{base}} \times \max(\text{MinimumDamageLimit}, 1.0 - (N_{\text{hits}} \times \text{DamageDecay}))$

Fórmula de Reducción de Hitstun (Hitstun Decay)

Fórmula: $\text{HitstunFrames}_{\text{actual}} = \max(\text{MinHitstun}, \text{BaseHitstun} \times (1.0 - (N_{\text{hits}} \times 0.04)))$

3.3. FSM de Personajes de Asistencia (Tag / Assist System)

```
enum class AssistState {
    OffscreenReady,
    EnteringScreen,
    ExecutingAttack,
    ExitingScreen,
    Cooldown
};

void UpdateAssistFSM(AssistCharacter& assist, PlayerCharacter& main_player) {
    switch (assist.state) {
        case AssistState::OffscreenReady:
            if (main_player.InputPressedAssist() && !main_player.IsInHitstun()) {
                assist.position = main_player.position - Vector2(3.0f * main_player.facing_dir, 0.0f);
                assist.state = AssistState::EnteringScreen;
            }
            break;

        case AssistState::EnteringScreen:
            assist.MoveToTargetLocation();
            if (assist.ReachedTarget()) {
                assist.ExecuteAssignedSpecialMove();
            }
    }
}
```

```

        assist.state = AssistState::ExecutingAttack;
    }
    break;

case AssistState::ExecutingAttack:
    // Regla "Happy Birthday": Si la asistencia es golpeada aquí,
    // recibe un 150% de daño multiplicado y entra en Hitstun en pantalla.
    if (assist.CheckHitboxCollision()) {
        assist.TakeDamage(multiplied_damage);
        break;
    }
    if (assist.IsAttackFinished()) {
        assist.state = AssistState::ExitingScreen;
    }
    break;

case AssistState::Cooldown:
    assist.cooldown_timer -= deltaTime;
    if (assist.cooldown_timer <= 0) {
        assist.state = AssistState::OffscreenReady;
    }
    break;
}
}
}

```

4. TEKKEN (3D Spatial Movement & Hit Levels)

4.1. Movimiento en Espacio 3D y Korean Backdash (KBD)

Mecánica de Ejes 3D

A diferencia del 2D, Tekken introduce el eje Z (profundidad) mediante las acciones de **Sidestep (SS)** y **Sidewalk (SW)**.

```

struct Transform3D {
    Vector3 position; // X = Horizontal, Y = Vertical (Salto/Agachado), Z = Profundidad del escenario
};

void Process3DMovement(Transform3D& t, const Input& input) {
    if (input.IsDoubleTapUp()) {
        t.position.z += sidestep_speed; // Paso lateral hacia el fondo
    } else if (input.IsDoubleTapDown()) {
        t.position.z -= sidestep_speed; // Paso lateral hacia el frente
    }
}
}

```

Lógica de Cancelación del Korean Backdash (KBD)

El KBD cancela el tiempo de recuperación de un *Backdash* ingresando la postura de agachado (**d/b** - Down/Back), lo que resetea la FSM del movimiento permitiendo encadenar desplazamientos atrás a alta velocidad.

```

Sequence Input: [Back, Back] ---> (Backdash State)
                |
                Input: [Down-Back] (Crouch Cancel)
                |
                v

```

```

Resets Backdash Recovery State
      |
      v
Sequence Input: [Back] -----> (Triggers New Backdash Instantly)

```

4.2. Matriz de Colisión por Niveles de Ataque (Hit Levels)

Nivel de Ataque	Bloqueo Parado (b)	Bloqueo Agachado (d/b)	Sin Bloqueo (Agachado)
High	Bloqueado (\$0\$ daño) Evadido completamente Evadido (Duck)		
Mid	Bloqueado (\$0\$ daño) Impacto Guard Break / Hit Impacto (Castiga Agachado)		
Low	Impacto (\$100\%\$ daño) Bloqueado (Parry/Block) Impacto (\$100\%\$ daño)		

4.3. Físicas de Juggle, Rebotadores y Colisión con Paredes

```

struct JugglePhysics {
    float air_gravity = -28.0f;
    int extension_mechanic_used = 0; // Tornado/Screw/Bound limitados a 1 por combo

    void ApplyAirHit(Character& victim, AttackProperties attack) {
        if (victim.is_airborne) {
            // Cada hit reduce la fuerza de elevación vertical
            float dampening = std::pow(0.85f, victim.combo_hit_count);
            victim.velocity.y = attack.launch_force_y * dampening;
            victim.velocity.x = attack.launch_force_x;

            // Verificación de extensión de combo (Ejemplo: Tornado)
            if (attack.has_tornado_property && extension_mechanic_used == 0) {
                victim.SetState(CharacterState::TornadoSpin);
                extension_mechanic_used++;
            }
        }
    }
};

```

Colisión con Paredes (Wall Splat System)

```

void CheckWallCollision(Character& victim, Stage3D& stage) {
    if (victim.is_in_hitstun && stage.IsTouchingWall(victim.position)) {

        if (victim.is_airborne) {
            // Impacto aéreo con la pared
            victim.SetState(CharacterState::WallSplat);
            victim.velocity = Vector3(0, -2.0f, 0); // Deslizamiento lento por la pared
            victim.hitstun_timer = 45; // Concede ventana de tiempo para un Wall Combo
        }
    }
}

```

5. RESUMEN COMPARATIVO DE ARQUITECTURA

| Motor / Mecánica | **Brawlhalla** | **Street Fighter** | **Dragon Ball FighterZ** | **Tekken** |

| :--- | :--- | :--- | :--- | :--- |

| **Dimensiones de Juego** | 2D Platform Fighter | 2D Classic / Footsies | 2D Anime / Tag Team | 3D Spatial Fighting |

| **Determinismo de Física** | Basado en impulsos (\$K\$) | Basado en *Frames* rígidos | Vectores + Tracking | Físicas de colisión 3D |

| **Defensa Principal** | Air Dodge / Ground Dodge | Guard (Atrás / Agachado) | Guard + Reflect (236S) | High/Mid/Low Guard + Sidestep |

| **Extensión de Combos** | True Combos vs. Strings | Links & Cancels | Magic Series + Assists | Juggle Physics + Wall Splat |